MedSync — Design Review Document

This document proposes the design of MedSync, a cloud-based web application for prescription and medication management, for the initial Design Review milestone; nothing described here is implemented yet.

1. Cover Page

Item	Details
Project title	MedSync — Smart Prescription & Medication Management System
Document type	Design Review Document (initial project milestone)
Project type	Academic project: System Design + RDBMS
Programme	B.Tech Computer Science
Team members	ShivaGanesh Varikuppala, Aanyadhar Dubey
Institution	Chandigarh University, UttarPradesh
Document version	1.0 (Design Review draft)
Date	18 October 2026
Status	Proposed design. No part of MedSync has been implemented or deployed.

Bracketed fields are placeholders to be filled in before submission.

2. Abstract / Executive Summary

MedSync is a proposed cloud-based web application that links doctors, patients and pharmacy staff through one prescription and medication-management platform, aiming to reduce missed doses, incorrect doses and medicines taken past their prescribed duration.

The design uses a **modular monolith** with a REST API, a normalised relational schema of 12 tables in **Azure SQL Database**, **Azure Blob Storage** for prescription documents, and **Power BI** for analytics. Security is addressed through role-based access control, password hashing, TLS, parameterised data access, secrets in a managed vault and audit logging, and is analysed through a nine-item threat model.

This document covers the design only. MedSync is a decision-support and record-keeping tool; it does not diagnose, prescribe autonomously or replace a doctor or pharmacist.

3. Table of Contents

1 Cover Page · 2 Abstract · 3 Contents · 4 Project Overview · 5 Functional Requirements · 6 Non-Functional Requirements · 7 User Roles and Feature Matrix · 8 System Architecture · 9 Database Design · 10 API Design · 11 Security Design · 12 Threat Model · 13 Design Tradeoffs · 14 Cloud and Analytics Design · 15 Future Scope · 16 Conclusion

4. Project Overview

4.1 Problem Statement

Medication errors in routine outpatient care have three common sources, each of which MedSync targets:

- **Patients** forget doses, take incorrect doses, or continue a medicine after its prescribed duration ends.
- **Doctors** need convenient access to a patient's previous prescriptions when writing a new one, and benefit from a reminder when two chosen medicines are known to interact.
- **Pharmacies** need a structured way to track stock, batches and expiry dates, so that expiring or low-stock medicines are noticed early.

4.2 Proposed Solution

MedSync is a proposed cloud-based web application that centralises prescriptions, medication schedules and pharmacy inventory behind one role-aware interface. It is organised into six application modules (authentication and authorisation, prescription management, medicine management, reminder management, pharmacy/inventory management, interaction checking) with audit logging as a cross-cutting concern, all exposed through a single REST API and backed by Azure SQL Database.

4.3 Objectives

1. Let doctors create structured prescriptions for a selected patient.
2. Give patients active prescriptions, a dose schedule, reminders and dose status tracking.
3. Keep a searchable medicine catalogue with informational interaction warnings.
4. Support pharmacy stock, batch and expiry tracking, and an analytics view.
5. Protect data with role-based access and audit logging, on a 3NF schema, within a scope two students can deliver.

4.4 Target Users

Role	Description	Primary needs
Doctor	Registered practitioner who issues prescriptions	Select a patient, create and manage prescriptions, review history, see interaction warnings
Patient	Person receiving prescriptions	View active and past prescriptions, follow the dose schedule, record dose status
Pharmacy Staff	Employee of a pharmacy	Maintain stock and batches, monitor expiry and low stock

All users are fictional or test accounts during the academic project; no real patient data will be used.

4.5 Project Scope

Core features: doctor prescription creation and management, patient prescription dashboard, medicine database and dosage management, medication reminders, prescription history, medicine interaction warnings. **Supporting features:** pharmacy inventory management, batch and expiry tracking, analytics dashboard.

Design assumptions made for this milestone, to be confirmed with the course faculty:

- **A-1 Account provisioning.** Patients self-register; doctor and pharmacy staff accounts are created by the team through a seeding script, because no administrator role is in scope.
- **A-2 Medicine catalogue.** The medicine catalogue and the interaction dataset are loaded from a curated sample dataset prepared by the team and are read-only through the application.
- **A-3 Doctor access to patients.** A doctor can search patients and view the prescription history of a patient they select; every such access is audit-logged. A formal doctor– patient care relationship is deferred to Future Scope.
- **A-4 Pharmacy staff access.** Pharmacy staff have no access to patient or prescription data in this version (least privilege), since dispensing is not in scope.
- **A-5 Reminder delivery.** Reminders are delivered in the application; email delivery is optional and not required.

4.6 Out of Scope

The initial academic version does **not** include medical diagnosis, autonomous prescription generation, emergency medical services, real-time video or audio doctor consultation, real-world payment processing, or autonomous medical decision-making.

MedSync is informational decision support and record-keeping software. Interaction warnings come from a curated dataset and do not replace the judgement of a doctor or pharmacist. No regulatory compliance or medical approval is claimed for this academic project.

5. Functional Requirements

FR-01 to FR-30 are listed by module; a range groups closely related requirements. "Shall" states design intent, not delivered functionality.

ID	Module	Requirement
FR-01	Authentication	Registered users can log in with their credentials.
FR-02	Authentication	Functionality depends on the user's role: Doctor, Patient or Pharmacy Staff.
FR-03	Authentication	Authenticated users can log out securely.
FR-04	Prescription	A doctor can select the patient for whom a prescription is created.
FR-05	Prescription	A doctor can create a prescription for a patient.
FR-06 to FR-10	Prescription	A doctor can add one or more medicines, specifying dosage, frequency, duration and instructions for each.
FR-11, FR-12	Prescription	Authorized users can view prescriptions they may access; previous prescriptions are kept as history.
ID	Module	Requirement
FR-13, FR-17	Prescription	A patient can view active prescriptions and previous prescriptions.
FR-14	Reminder	A medication schedule is generated from dosage, frequency, start date and duration.

FR-15	Reminder	Reminders are provided for scheduled doses.
FR-16	Reminder	A patient can mark a reminder Taken, Skipped or Pending.
FR-18 to FR-20	Medicine	The system keeps a medicine database; authorized users can search it and see information on a selected medicine.
FR-21, FR-22	Interaction	Selected medicines are checked against a configured interaction dataset; a known interaction shows an informational warning to the doctor.
FR-23 to FR-25	Pharmacy	Pharmacy users can view inventory, add stock and update quantities.
FR-26, FR-27	Pharmacy	The system keeps batch information and identifies medicines approaching expiry.
FR-28	Pharmacy	The system identifies medicines whose stock is below a defined threshold.
FR-29, FR-30	Analytics	Application data is available for analytics, and an analytics view shows prescription trends, medicine usage, inventory levels, low-stock and expiring medicines.

6. Non-Functional Requirements

The 2-second target is a design goal to be measured later, not a measured result.

ID	Requirement	Proposed design response (section)
NFR-01 Security	Protect user and prescription data from unauthorized access	Authentication, RBAC, TLS, hashing, parameterised queries, secrets in a vault (11)
ID	Requirement	Proposed design response (section)

NFR-02 Performance	95% of normal API requests within 2 seconds under expected project load	Indexes, pagination, small payloads, reminders pregenerated (9.6, 10.6)
NFR-03 Availability	Cloud-hosted app stays accessible in normal conditions	Managed Azure services, health check (14)
NFR-04 Scalability	Growth in users and records without complete redesign	Stateless API, 3NF schema, modular boundaries, resizable services (8, 14)
NFR-05 Reliability	Reliable storage; no inconsistent or incomplete transactions	One transaction per multi-step write; automated backups (8.3, 14)
NFR-06 Data Integrity	Consistent relationships among all entities	Keys, UNIQUE and CHECK constraints, transactions (9)
NFR-07 Usability	Simple, role-specific interface	A separate dashboard per role (7)
NFR-08 Maintainability	Components can change without affecting the whole system	Modular monolith with moduleowned tables (8.2)
NFR-09 Privacy	Information visible only to authorized users	Ownership checks, least privilege, de-identified analytics (11.2, 14)
NFR-10 Auditability	Prescription and inventory changes are logged	Append-only audit_logs written in the same transaction (9.4, 11.7)

7. User Roles and Feature Matrix

Access is granted per role, then narrowed by ownership: a Patient sees only their own records, a Doctor sees prescriptions they issued and the history of a selected patient, and Pharmacy Staff see only their own pharmacy's inventory. In the matrix, "Own" means records linked to the signed-in user and "—" means no access.

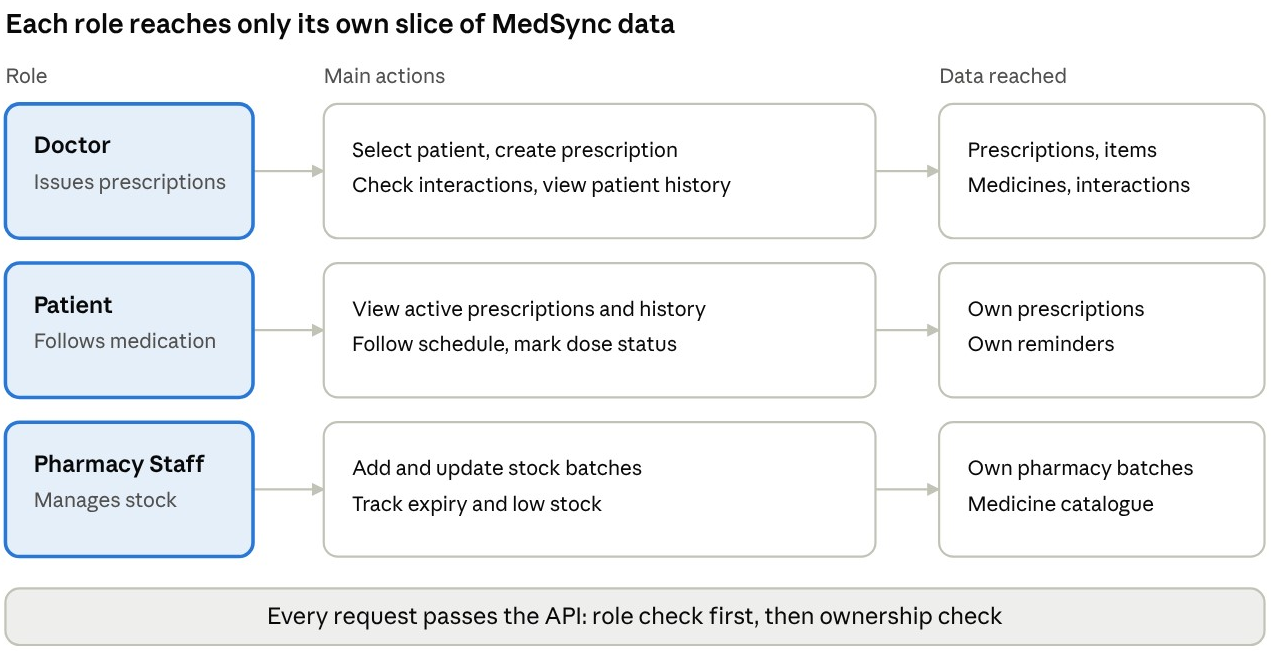


Figure 1: role interaction · 3 roles

Figure 1: User and role interaction. Each role has its own actions and data slice; the API applies a role check and then an ownership check on every request.

Feature	Doctor	Patient	Pharmacy Staff
Log in / log out (FR-01, FR-03)	Yes	Yes	Yes
Search and select a patient (FR04)	Yes (identity fields only)	—	—
Create and manage prescriptions (FR-05 to FR-10)	Create; edit own while Active	—	—
View a prescription (FR-11)	Own issued; selected patient's history	Own	—
Prescription history (FR-12, FR-17)	Selected patient's history	Own	—
Active prescription dashboard (FR-13)	—	Own	—
Medication schedule and reminders (FR-14, FR-15)	—	Own	—

Mark reminder Taken / Skipped / Pending (FR-16)	—	Own	—
Feature	Doctor	Patient	Pharmacy Staff
Search medicines and view information (FR-18 to FR-20)	Yes	Yes (read)	Yes
Interaction check and warning (FR-21, FR-22)	Yes	—	—
View inventory and batches (FR23, FR-26)	—	—	Own pharmacy
Add / update stock (FR-24, FR-25)	—	—	Own pharmacy
Expiry and low-stock lists (FR-27, FR-28)	—	—	Own pharmacy
Analytics view (FR-29, FR-30)	Prescription and medicineusage pages	—	Inventory pages

8. System Architecture

8.1 Architecture Overview

MedSync is proposed as a **modular monolith**: one deployable application whose code is divided into modules with clear responsibilities and boundaries. Users reach a browserbased frontend, which calls a backend REST API; the API routes each request to an application module, and modules read and write a single Azure SQL Database. Azure Blob Storage holds prescription documents, and Power BI reads de-identified reporting views for analytics. Figure 2 shows the logical layout.

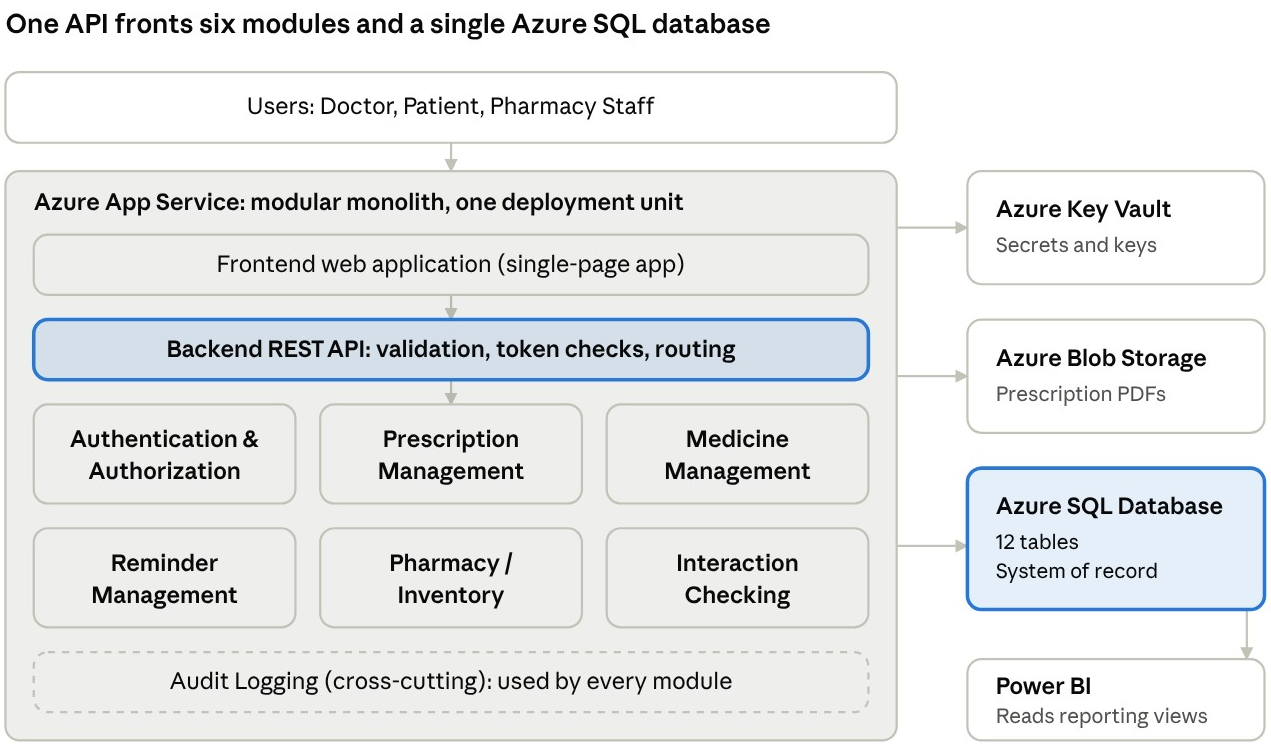


Figure 2: high-level system architecture · 6 modules, 3 Azure data services

Figure 2: High-level system architecture. Users reach the frontend, which calls the REST API; the API routes to six modules, all inside one App Service. Modules use Azure SQL for data, Blob Storage for PDFs and Key Vault for secrets; Power BI reads reporting views.

8.2 Major Components

Module	Responsibility	Tables owned	Requirements
Authentication & Authorization	Registration, login, logout, token, role and ownership checks	users, doctors, patients	FR-01 to FR-03, FR-11
Prescription Management	Prescriptions, items, history, PDF generation	prescriptions, prescription_items	FR-04 to FR-13, FR-17
Medicine Management	Catalogue, search, medicine information	medicines	FR-18 to FR-20
Module	Responsibility	Tables owned	Requirements

Interaction Checking	Compares selected medicines with the curated dataset; informational warnings	medicine_interactions	FR-21, FR-22
Reminder Management	Dose schedule, due reminders, Taken / Skipped / Pending	reminders	FR-14 to FR-16
Pharmacy / Inventory Management	Stock, batches, expiry and low stock per pharmacy	pharmacies, pharmacy_staff, medicine_batches	FR-23 to FR-28
Audit Logging (cross-cutting)	Append-only records of important actions	audit_logs	NFR-10

Module rule: a module changes only the tables it owns and reaches another module's data through that module's service interface, which keeps NFR-08 achievable in one codebase. The frontend calls only the REST API; Azure SQL, Blob Storage and Power BI are described in Section 14.

8.3 Data Flow

Figure 3 traces the two key flows: prescription creation (FR-04 to FR-10, FR-14, FR-21, FR22) and the patient reminder cycle (FR-15, FR-16).

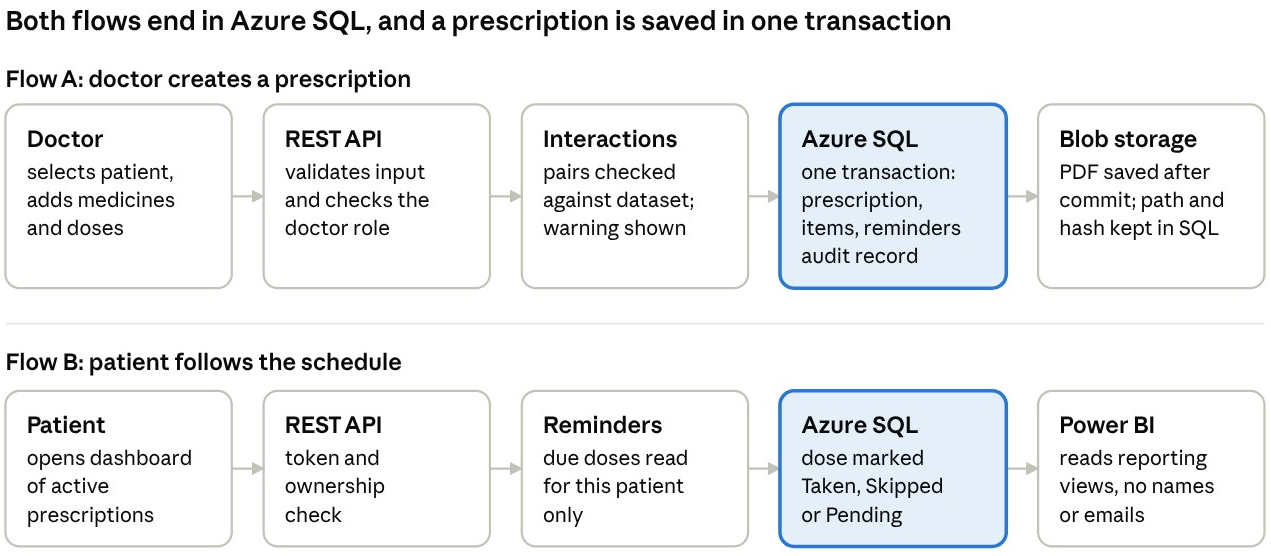


Figure 3: data flow · 2 flows, 5 steps each

Figure 3: Data flow. Flow A shows prescription creation, saved in one database transaction with the PDF written afterwards. Flow B shows the patient reminder cycle and the deidentified path to Power BI.

Flow A: Doctor creates a prescription

1. The doctor signs in, selects a patient, and adds medicines with dosage, frequency, duration and instructions (FR-04 to FR-10).
2. The medicine picker queries the catalogue; the interaction check shows any known pair as an informational warning (FR-21, FR-22).
3. On submit, the API validates input and confirms the Doctor role.
4. One database transaction inserts the prescription and items, generates the dose reminders (FR-14) and writes an audit record; any failure saves nothing (NFR-05).
5. After commit, a PDF copy is stored in Blob Storage and its path and SHA-256 hash are saved on the prescription.

Flow B: Patient follows the schedule

1. The patient signs in and opens the dashboard of active prescriptions (FR-13), which lists due and upcoming reminders (FR-15).
2. The patient marks a dose Taken, Skipped or Pending; status and timestamp are saved (FR-16).
3. De-identified reporting views later feed Power BI (FR-29).

8.4 Technology Stack

Technologies are **proposed choices** for the implementation phase and may change if a team constraint requires it.

Layer	Proposed technology	Reason
Frontend	React with TypeScript (singlepage app)	Component-based; suited to role dashboards
Backend API	ASP.NET Core Web API (C#)	Azure and SQL Server integration; built-in validation and authorization
Data access	Entity Framework Core	Parameterised queries; migrations
Database	Azure SQL Database	Managed relational engine with constraints, transactions, backups

File storage	Azure Blob Storage (private container)	Durable, low-cost PDF storage
Layer	Proposed technology	Reason
Analytics	Power BI	Dashboards over SQL reporting views
Hosting	Azure App Service (API and built frontend together, so cookies stay same-site)	One deployment unit
Secrets and monitoring	Azure Key Vault with managed identity; Application Insights	No credentials in code; request and error telemetry

8.5 Requirements Traceability

Key endpoints per requirement group; full endpoint lists are in Section 10 and table definitions in Section 9.

Requirements	Module	Key endpoints
FR-01 to FR-03	Authentication & Authorization	POST /auth/login, POST /auth/logout
FR-02, FR-11	Authentication (role and ownership checks)	Enforced on every endpoint
FR-04 to FR-10	Prescription Management	GET /patients, POST /prescriptions
FR-11 to FR-13, FR-17	Prescription Management	GET /prescriptions/{id}, GET /patients/{patientId}/prescriptions, GET /me/prescriptions
FR-14 to FR-16	Reminder Management	GET /me/reminders, PATCH /reminders/{id}
FR-18 to FR-20	Medicine Management	GET /medicines, GET /medicines/{id}
FR-21, FR-22	Interaction Checking	POST /interaction-checks

FR-23 to FR-28	Pharmacy / Inventory	/pharmacy/inventory, /pharmacy/batches (GET, POST, PATCH), /pharmacy/batches/expiring, /pharmacy/inventory/low-stock
FR-29, FR-30	Power BI over reporting views	None; Power BI reads views directly
NFR-10	Audit Logging	Written by state-changing endpoints

9. Database Design

The relational schema is proposed for Azure SQL Database and is reviewed rather than copied from the starting list of ten tables. Two tables were added and a few attributes reshaped so the model supports every functional requirement and holds in third normal form (3NF): `pharmacy_staff` (the starting list had no way to tie pharmacy staff to a pharmacy) and `audit_logs` (NFR-10). The final design has **12 tables**, slightly above the "approximately 8–10" guideline, because both additions are needed for correctness.

9.1 Entities

Identity: users, doctors, patients. Pharmacy: pharmacies, pharmacy_staff, medicine_batches. Medicines: medicines, medicine_interactions. Prescriptions: prescriptions, prescription_items, reminders. Audit: audit_logs.

9.2 Relationships

Relationship	Cardinality
users → doctors / patients / pharmacy_staff	one to zero-or-one (profile matches the user's role)
pharmacies → pharmacy_staff	one to many
doctors → prescriptions; patients → prescriptions	one to many each
prescriptions → prescription_items	one to many (at least one item, enforced by the application)
medicines → prescription_items	one to many
prescription_items → reminders	one to many

medicines ↔ medicines through medicine_interactions	many to many; each pair stored once
pharmacies ↔ medicines through medicine_batches	many to many
users → audit_logs	one to many

9.3 ER Diagram

Figure 4 shows the 12 entities with primary keys (PK), foreign keys (FK) and cardinalities. Only the key columns and one or two identifying attributes appear, to keep it readable; full column lists are in Section 9.4.

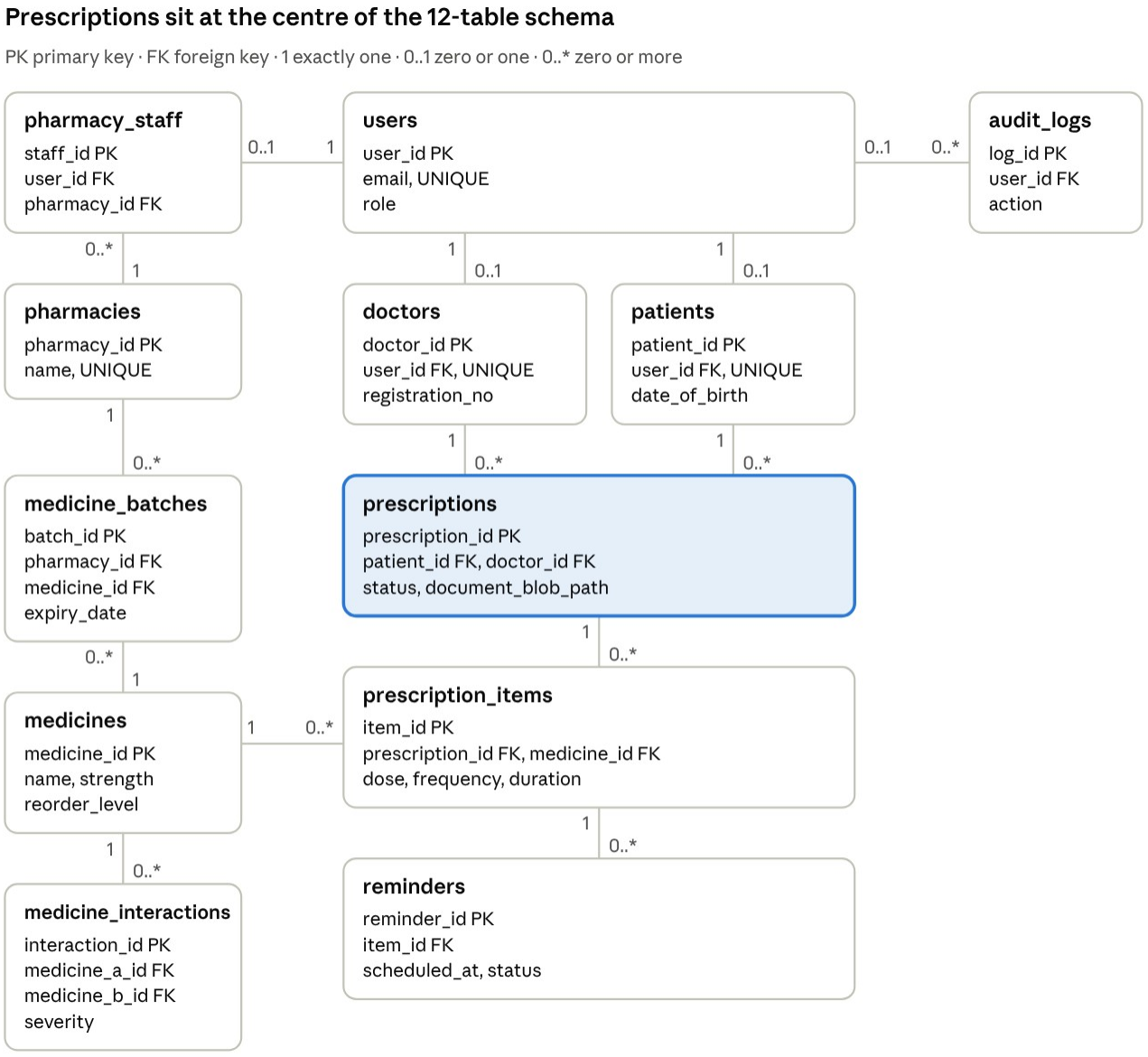


Figure 4: ER diagram · 12 entities, 13 relationship lines

Figure 4: Entity-relationship diagram. One line is drawn between medicine_interactions and medicines, but both medicine_a_id and medicine_b_id reference medicines. audit_logs.user_id references users; entity_type and entity_id in audit_logs identify the changed record without a foreign key, because the log is append-only and covers several tables.

9.4 Tables

Types follow SQL Server conventions: INT IDENTITY keys, NVARCHAR text, DATE and DATETIME2 (UTC) dates, BIT flags, DECIMAL(6,2) dose quantity. NN means NOT NULL; "today" and "now" mean the current UTC date and time.

Table	Columns and constraints
users	user_id PK; email NN UNIQUE; password_hash NN; full_name NN; phone; role NN CHECK (DOCTOR, PATIENT, PHARMACY_STAFF); is_active NN DEFAULT 1; created_at NN DEFAULT now
doctors	doctor_id PK; user_id FK NN UNIQUE; registration_no NN UNIQUE; specialization NN
patients	patient_id PK; user_id FK NN UNIQUE; date_of_birth NN CHECK (after 1900-01-01); gender CHECK (MALE, FEMALE, OTHER)
pharmacies	pharmacy_id PK; name NN UNIQUE; address NN; phone
pharmacy_staff	staff_id PK; user_id FK NN UNIQUE; pharmacy_id FK NN; designation
medicines	medicine_id PK; name, generic_name, strength NN; dosage_form NN CHECK (TABLET, CAPSULE, SYRUP, INJECTION, DROPS, OINTMENT, OTHER); description; precautions; reorder_level NN DEFAULT 20 CHECK (>= 0); is_active NN DEFAULT 1; UNIQUE (name, strength, dosage_form)
medicine_interactions	interaction_id PK; medicine_a_id FK NN; medicine_b_id FK NN; severity NN CHECK (MINOR, MODERATE, MAJOR); description NN; CHECK (medicine_a_id < medicine_b_id); UNIQUE (medicine_a_id, medicine_b_id)

Table	Columns and constraints
prescriptions	prescription_id PK; patient_id FK NN; doctor_id FK NN; prescribed_on NN DEFAULT today; notes; status NN DEFAULT ACTIVE CHECK (ACTIVE, CANCELLED); document_blob_path; document_sha256 CHAR(64); created_at, updated_at NN DEFAULT now
prescription_items	item_id PK; prescription_id FK NN; medicine_id FK NN; dose_quantity NN CHECK (> 0); dose_unit NN CHECK (TABLET, CAPSULE, ML, DROPS, APPLICATION, UNIT); frequency_per_day NN CHECK (1 to 6); duration_days NN CHECK (1 to 365); start_date NN DEFAULT today; instructions; UNIQUE (prescription_id, medicine_id)
reminders	reminder_id PK; item_id FK NN; scheduled_at NN; status NN DEFAULT PENDING CHECK (PENDING, TAKEN, SKIPPED); status_updated_at; UNIQUE (item_id, scheduled_at)
medicine_batches	batch_id PK; pharmacy_id FK NN; medicine_id FK NN; batch_number NN; received_date NN DEFAULT today; expiry_date NN; quantity_on_hand NN DEFAULT 0 CHECK (>= 0); UNIQUE (pharmacy_id, medicine_id, batch_number); CHECK (expiry_date > received_date)
audit_logs	log_id PK (BIGINT); user_id FK (nullable); action NN; entity_type NN; entity_id; details (JSON of old and new values); ip_address; created_at NN DEFAULT now

9.5 Primary and Foreign Keys

Every table has a single-column surrogate integer primary key. The 14 foreign keys are the columns marked FK in Section 9.4. None uses ON DELETE CASCADE: clinical and audit records are never physically deleted and users are deactivated through `is_active`, so a delete that would orphan a row is rejected (NO ACTION).

9.6 Constraints

Constraint	Where it is used
PRIMARY KEY	All 12 tables
FOREIGN KEY	The 14 foreign keys in Section 9.4; referential integrity (NFR-06)
NOT NULL	Identifying and required business columns
Constraint	Where it is used
UNIQUE	Email; one profile per user; medicine (name, strength, form); interaction pair; one item per medicine per prescription; one reminder per item and time; one batch per pharmacy, medicine and batch number
CHECK	Allowed-value lists; positive dose; frequency 1 to 6; duration 1 to 365 days; quantity and reorder level at least 0; expiry after receipt; interaction pair ordered
DEFAULT	Status values, active flags, quantity 0, reorder level 20, dates and timestamps
Transactions	Prescription + items + reminders + audit record; batch change + audit record (NFR-05)

Proposed indexes (NFR-02): prescriptions by patient and date and by doctor and date; prescription_items by prescription; reminders by status and scheduled time; medicines by name and generic name; medicine_batches by pharmacy and expiry date; audit_logs by entity and by time. Rules a CHECK cannot express (a user's role must match their profile table; a prescription needs at least one item) are enforced and tested in the application.

9.7 Normalization / 3NF

1NF. Every column holds one atomic value. Medicines of a prescription are rows in a list inside prescri medicines are one pair per row.

2NF. Each table has a single-column surrogate key, so no non-key column can depend on only part of a key. The natural composite keys are kept as UNIQUE constraints (for example prescription_id + medicine_id) to prevent duplicates.

3NF. No non-key column depends on another non-key column. The design decisions that achieve this are:

Decision	Reason
Doctor, patient and staff details in separate tables joined to users	Avoids many role-specific nullable columns
prescriptions and prescription_items store ids, not names or medicine details	Those attributes depend on the patient, doctor or medicine
Decision	Reason
End date, "active" state and remaining days are derived from start_date + duration_days	A stored end date would depend on two other columns
reminders stores item_id only	The patient is reachable through the item and prescription
Stock per pharmacy and medicine is the SUM of batch quantities	A stored total could contradict the batches
reorder_level sits on medicines; each interaction pair is stored once	It depends on the medicine; prevents contradictory (A, B) and (B, A) rows

Two deliberate exceptions are documented. `audit_logs.details` holds a JSON snapshot of old and new values, which is a point-in-time record rather than a normalised attribute. `prescriptions.document_sha256` and `document_blob_path` describe a generated file and are stored with the prescription it belongs to, which is a one-to-one relationship.

An "active" prescription is one with status ACTIVE that still has at least one item whose start_date + duration_days is not yet past. All other prescriptions appear in history.

10. API Design

The proposed REST API is versioned under `/api/v1`, uses JSON over HTTPS and is grouped by module; endpoints are design proposals. Identity always comes from the authenticated token, never from the request body, and state-changing endpoints write an audit record (Section 11.7).

10.1 Authentication APIs (Authentication & Authorization module)

Endpoint	Role	Purpose (requirement)
POST /auth/register	Public (patients)	Patient self-registration (FR-01)

POST /auth/login	Public	Verify credentials, start session (FR-01)
POST /auth/refresh	Signed-in	Issue a new short-lived token (FR-01)
POST /auth/logout	Signed-in	End session, clear cookie (FR-03)
GET /auth/me	Signed-in	Current user and role (FR-02)

10.2 Prescription APIs (Prescription Management module)

Endpoint	Role	Purpose (requirement)
GET /patients?search=	Doctor	Search patients; identity fields only (FR-04)
POST /prescriptions	Doctor	Create a prescription with items; creates reminders in the same transaction (FR-05 to FR-10, FR-14)
GET /prescriptions/{id}	Doctor (issuer), Patient (own)	View one prescription (FR-11)
GET /doctors/me/prescriptions	Doctor	Prescriptions issued by the doctor (FR-11, FR-12)
GET /patients/{patientId}/prescriptions	Doctor	History of a selected patient, audit-logged (FR-12)
GET /me/prescriptions?status=active|history	Patient	Active or previous prescriptions (FR-13, FR-17)
PATCH /prescriptions/{id}	Doctor (issuer, ACTIVE)	Correct dose, frequency, duration or instructions; regenerates pending reminders
POST /prescriptions/{id}/cancel	Doctor (issuer)	Set status to CANCELLED
GET /prescriptions/{id}/document	Doctor (issuer), Patient (own)	Stream the stored PDF after an ownership check (FR-11)

10.3 Medicine APIs (Medicine Management and Interaction Checking modules)

Endpoint	Role	Purpose (requirement)

GET /medicines?q=&page=	All roles	Search the catalogue (FR-18, FR-19)
GET /medicines/{id}	All roles	Medicine details (FR-20)
POST /interaction-checks	Doctor	Medicine ids in; known pairs with severity and description out, as informational warnings (FR-21, FR-22)

10.4 Reminder APIs (Reminder Management module)

Endpoint	Role	Purpose (requirement)
GET /me/reminders?from=&to=&status=	Patient	Medication schedule for a date range (FR-14)
GET /me/reminders/due	Patient	Reminders due now or overdue (FR15)
PATCH /reminders/{id}	Patient (own)	Set TAKEN, SKIPPED or PENDING (FR-16)

10.5 Pharmacy APIs (Pharmacy / Inventory Management module)

The pharmacy is always taken from the signed-in staff member's pharm ver from a URL or body parameter.

Endpoint	Role	Purpose (requirement)
GET /pharmacy/inventory	Pharmacy Staff	Stock per medicine for the own pharmacy (FR-23)
GET /pharmacy/batches?medicineId=	Pharmacy Staff	Batches with number, expiry and quantity (FR-26)
POST /pharmacy/batches	Pharmacy Staff	Add stock as a new batch (FR-24, FR-26)
PATCH /pharmacy/batches/{id}	Pharmacy Staff	Update quantity or batch details (FR-25)
Endpoint	Role	Purpose (requirement)

GET /pharmacy/batches/expiring?days=60	Pharmacy Staff	Batches expiring in the window; 60 days is a proposed default (FR-27)
GET /pharmacy/inventory/low-stock	Pharmacy Staff	Stock below reorder_level, default 20 (proposed) (FR-28)

Analytics (FR-29, FR-30). No analytics endpoints are proposed. Power BI reads deidentified reporting views directly from Azure SQL (Section 14), which keeps analytics queries away from the transactional API. 10.6 API Security

Control	Design
Transport	HTTPS only; HSTS
Authentication	Token in an HttpOnly, Secure, SameSite cookie on every endpoint except register and login
Authorization	Role check, then an ownership check on the specific record
Input and injection	Server-side validation; unknown fields rejected; ORM parameterised queries
Abuse	Rate limits on login and register; anti-forgery header on state-changing requests
Errors and performance	Generic error messages; pagination (page size 20) and indexed queries (NFR-02)

11.Security Design

Security is layered so no single failed control exposes prescription data. All controls are proposed; no formal certification or regulatory compliance is claimed for this academic project.

11.1 Authentication

- Users log in with email and password (FR-01); passwords are stored only as salted hashes (Argon2id or bcrypt), with a minimum length of 10.
- On success the API issues a short-lived signed token (proposed 15 minutes) in an HttpOnly, Secure, SameSite=Strict cookie; /auth/refresh e sion and logout clears the cookie (FR-03).
- Login responses are identical for an unknown email and a wrong password; login is rate-limited and failures are written to audit_logs .

11.2 Authorization / RBAC

Every request gets a **role check** (may this role call this endpoint?) and an **ownership check** (may this user see this record?). The second prevents horizontal privilege escalation, such as a patient changing an id to read another patient's prescription. The role is read from the server-side user record, never from client input.

Role	Allowed	Ownership rule
Doctor	Patient search; prescription create, edit, cancel; history of a selected patient; medicine search; interaction check; prescription and medicine-usage analytics	Edits only own prescriptions; each history view is audit-logged (A-3)
Patient	Own prescriptions, schedule and reminders; medicine search	Every query filtered by the patient id from the token
Pharmacy Staff	Inventory, batches, expiry and low stock of own pharmacy; medicine search; inventory analytics	Pharmacy id from the pharmacy_staff record; no patient data (A-4)

11.3 Data Protection

Concern	Design
In transit	TLS 1.2 or higher for browser-to-API, API-todatabase and API-to-Blob traffic
At rest	Azure SQL transparent data encryption; Blob Storage service-side encryption (Azure defaults)
Minimisation	Only needed fields stored; analytics views exclude names, emails and dates of birth
Logs	No passwords, tokens or full prescriptions in logs or URLs
Concern	Design
Integrity and test data	SHA-256 of each stored PDF checked when served; only fictional test data

11.4 Input Validation

- Server-side validation of type, length, range and allowed values, matching the database CHECK constraints (dose > 0, frequency 1 to 6, duration 1 to 365 days).
- ORM parameterised queries only; free-text fields are shown as text, never HTML; a Content-Security-Policy header limits script sources.
- Users cannot upload files; PDFs are generated by the server only.

11.5 Secret Management

- Connection settings, signing keys and storage credentials live in Azure Key Vault and are read at runtime through a managed identity; where possible SQL and Blob also use that identity, so no database password or storage key exists to leak.
- No secrets in source control; a secret scanner runs on commits and local settings files are git-ignored.
- Separate secrets for development and demonstration; keys can be rotated without code changes.

11.6 Database Security

Control	Design
Network	Firewall denies all by default; only the application's addresses (or a private endpoint) and temporary developer addresses are allowed; "Allow Azure services" is not used
Accounts	Application account: SELECT, INSERT, UPDATE on application tables, no DELETE on clinical tables, INSERT and SELECT only on audit_logs. Separate migration account for schema changes. Analytics account: SELECT on reporting views only
Blob container	Private; anonymous access disabled; reachable only through the API
Backups	Azure SQL automated backups with the platform's default retention

11.7 Audit Logging

The a e is append-only for the application account, and each record is written in the same transaction as the change it describes. Logged events: prescription created, edited (old and new values in details) and cancelled; a do g a patient's history; batch added or changed

(old and new quantity); failed logins. Each record stores the acting user, entity, time and client address; review is manual in this version.

12. Threat Model

Nine threats, their attack surface and proposed mitigation. Likelihood (L) and impact (I) are qualitative estimates for an internet-facing prototype holding only test data. Figure 5 shows where each threat is mitigated.

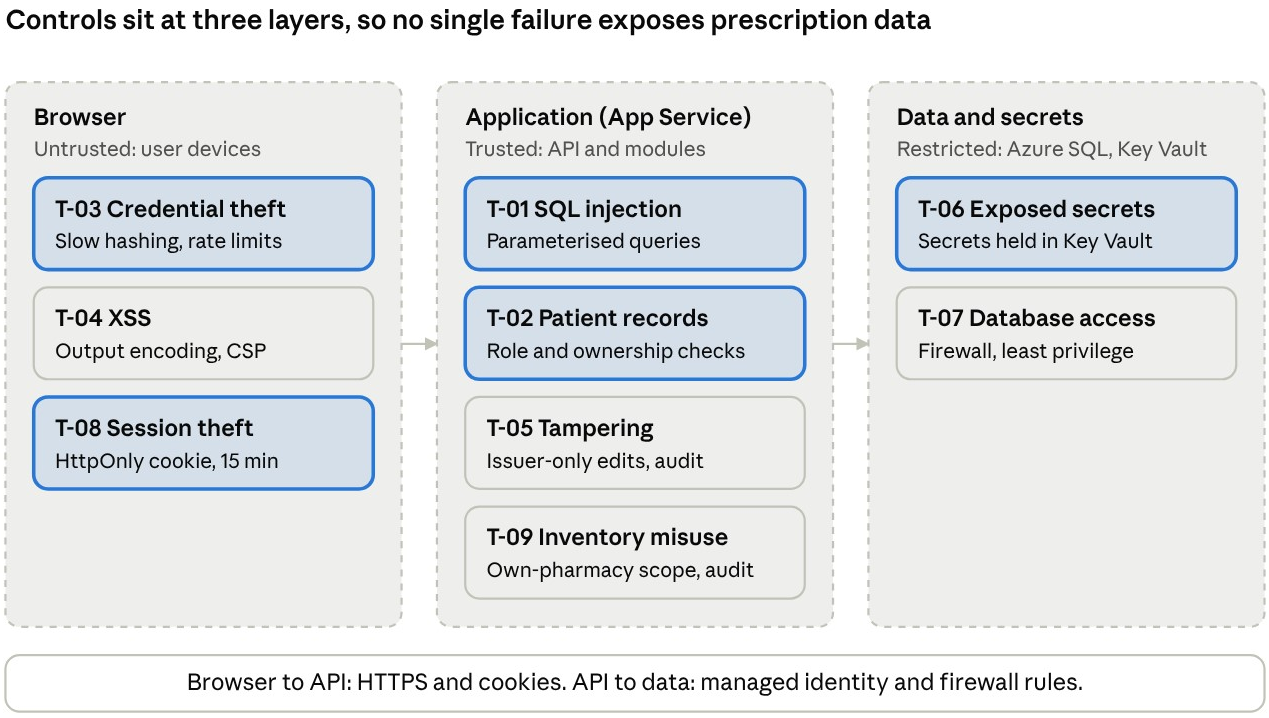


Figure 5: security and threat model overview · 3 layers, 9 threats

Figure 5: Security and threat model overview. Dashed zones are trust boundaries; each threat from the table below appears at the layer where its main control operates.

ID	Threat	Attack surface	Potential impact	Likelihood / risk	Proposed mitigation
T-01	SQL injection	Search boxes, login, query parameters, request bodies	Data theft or alteration	L medium, I high: **High**	ORM parameterised queries; validation; least-privilege database account (11.4, 11.6)

T-02	Unauthorized access to patient records	Prescription and history endpoints; changed ids	Exposure of health information	L high, I high: **High**	Role and ownership check on every record; patient id from token; 404 for records not owned; audit of history views (11.2, 11.7)
T-03	Credential theft	Login form, phishing, brute force	Account takeover; misuse of a doctor account	L medium, I high: **High**	Slow salted hashing; length rule; rate limits; identical failure messages; failed-login audit (11.1)
ID	Threat	Attack surface	Potential impact	Likelihood / risk	Proposed mitigation
T-04	Cross-site scripting	Free-text fields: instructions, notes, descriptions	Script runs in another user's browser	L medium, I medium: **Medium**	Output encoding; text-only rendering; CSP; HttpOnly cookies (11.4)
T-05	Prescription tampering	Edit endpoint, direct database writes, stored PDF	Altered dose or duration; patient harm	L low, I high: **MediumHigh**	Issuer-only edits while ACTIVE; old and new values audited; PDF hash check; no DELETE on clinical tables (11.3, 11.7)
T-06	Exposed database or API secrets	Repository, config files, logs, error pages	Direct database or storage access	L medium, I high: **High**	Key Vault with managed identity; secret scanning; no secrets in logs; rotation (11.5)
T-07	Unauthorized database access	Public endpoint, stolen connection details	Bulk theft or modification	L low, I high: **MediumHigh**	Firewall deny-bydefault; managed identity; separate accounts (11.6)
T-08	Session or token theft	Network interception, XSS, stolen device, logs	Attacker acts as the user until expiry	L medium, I high: **High**	TLS and HSTS; HttpOnly Secure SameSite cookie; 15minute life; CSRF header; no tokens in URLs or logs (10.6, 11.1)

T-09	Malicious or incorrect inventory change	Batch endpoints; data-entry mistakes	Wrong stock or expiry; false alerts	L medium, I medium: **Medium**	Pharmacy id from staff record; CHECK constraints; audit with old and new quantity (10.5, 11.7)

Residual risks accepted for the academic version: no server-side token revocation, no multi-factor authentication, manual review of audit logs, and unrestricted doctor search across patients (assumption A-3). Each is listed in Future Scope.

13. Design Trade-offs

Each decision compares MedSync's choice with its main alternative and justifies it for a two-member academic project.

13.1 Modular Monolith vs Microservices

	Modular monolith (chosen)	Microservices
Advantages	One codebase and deployment; ordinary transactions across modules; low cost; easy debugging	Independent deployment and scaling; fault isolation
Disadvantages	Whole app deploys and scales as one	Network failures, distributed transactions, many pipelines and monitors

Decision. MedSync uses a modular monolith. Creating a prescription must save the prescription, items, reminders and audit record atomically (NFR-05), which is one ordinary transaction in one application and one database. Two students would otherwise spend their time on service plumbing instead of the schema and security work; moduleowned tables and service interfaces (Section 8.2) keep a later split possible (NFR-04, NFR-08).

13.2 Azure SQL (Relational) vs NoSQL for Transactional Prescription Data

	Azure SQL (chosen)	NoSQL document store
Advantages	Foreign keys, CHECK constraints, ACID multi-row transactions; joins and aggregates for history, inventory and analytics; direct Power BI connection	Flexible structure; easy horizontal scaling

Disadvantages	Schema changes need migrations; scales up before out	Weaker built-in referential integrity; limited joins and crossdocument transactions; integrity rules move into application code

Decision. MedSync uses Azure SQL. The data is strongly relational and correctness matters: an item must reference a real medicine, a batch quantity must never be negative, and a prescription needs its patient. Constraints deliver NFR-06 directly, the RDBMS learning goals (3NF, keys, constraints) need a relational engine, and expected volumes are too small to need NoSQL scaling.

13.3 Blob Storage vs Large Documents in the Relational Database

	Blob Storage (chosen)	Documents in the database (VARBINARY)
Advantages	Cheap and durable; small database that backs up quickly; separate access control	One store; transactional with the owning row
Disadvantages	Two systems to secure; a missing or orphaned file is possible	Larger, costlier database; slower backups; large reads compete with transactions

Decision. Prescription PDFs go to Blob Storage, with `document_blob_path` and `document_sha256` on the prescription row. The files are never queried relationally, and the structured data in the database stays the authoritative record, so a missing document is simply regenerated. For two students it is a few lines of integration and avoids oversizing the database.

14. Cloud and Analytics Design

MedSync is proposed for managed Azure services so the team spends its effort on the application. **App Service** hosts the API and built frontend (HTTPS only; health check for NFR-03). **Azure SQL Database** is the system of record (firewall deny-by-default, automated backups, resizable tier for NFR-04 and NFR-05). **Blob Storage** holds prescription PDFs in a private container reached through a managed identity. **Key Vault** holds secrets, and **Application Insights** measures the 2-second target (NFR-02). **Power BI** connects through a read-only analytics account to reporting views that contain no names, emails, dates of birth or free-text notes (NFR-09), keeping analytics queries away from the transactional API.

Analytics design (FR-29, FR-30)

Reporting view	Content	Dashboard visual

vw_prescription_trends	Prescriptions and items per week and month, by status	Prescription trend (doctors)
vw_medicine_usage	Prescription items per medicine	Most-prescribed medicines (doctors)
vw_inventory_levels	Summed stock per pharmacy and medicine	Inventory levels (pharmacy staff)
Reporting view	Content	Dashboard visual
vw_low_stock	Medicines below reorder_level	Low-stock list (pharmacy staff)
vw_expiring_batches	Batches expiring within the configured window	Expiring list (pharmacy staff)

The report has two pages, Prescription Analytics and Inventory Analytics, linked from the matching role's dashboard (FR-30). Open point for faculty review: access is limited to named project and demonstration accounts, and row-level security per pharmacy is deferred to Future Scope.

15. Future Scope

Items below are not part of the initial academic version.

- **Security:** multi-factor authentication; server-side token revocation; automated auditlog alerts.
- **Privacy:** explicit doctor–patient care relationship; patient consent for record sharing.
- **Administration:** administrator role for account and catalogue management.
- **Pharmacy:** prescription verification and dispensing workflow; per-pharmacy reorder levels.
- **Interactions:** ingredient-level data; allergy checks; a licensed clinical dataset.
- **Reminders and analytics:** email or push notifications; adherence reports; row-level security per pharmacy; embedded Power BI.
- **Architecture and quality:** splitting a module into a service if load requires it; automated load testing.

16. Conclusion

This Design Review proposes MedSync as a modular monolith with a REST API, a 12-table schema in third normal form on Azure SQL Database, Blob Storage for prescription documents and Power BI for analytics. Requirements FR-01 to FR-30 and NFR-01 to NFR10 are traced to modules, tables

and endpoints, and the threat model addresses nine risks with concrete controls. The scope is limited to what two students can build and defend: informational interaction warnings only, with no diagnosis, autonomous prescribing, payment or consultation features. As the design is proposed and not implemented, the next steps are to confirm assumptions A-1 to A-5 with the faculty, create the schema and seed data, and implement the modules in order of core-feature priority.